

1) Create a NumPy array containing numbers from 10 to 100 with a step of 10.

```
import numpy as np

arr1 = np.arange(10, 101, 10)
print(arr1)

# or

arr2 = np.linspace(10, 100, 10, dtype=int)
print(arr2)

# We can perform this operation by using arange(), linspace()
# np.arange() -> Values with a specific step
# np.linspace() -> Values with a specific number of elements

[ 10  20  30  40  50  60  70  80  90 100]
[ 10  20  30  40  50  60  70  80  90 100]
```

2) Create a 3 x 4 array of zeros. This can be used to perform by using np.zeros(), np.full

```
arr1 = np.zeros((3, 4), dtype=int)
print(arr1)

# or

arr2 = np.full((3, 4), 0)
print(arr2)

[[0 0 0 0]
 [0 0 0 0]
 [0 0 0 0]]
[[0 0 0 0]
 [0 0 0 0]
 [0 0 0 0]]
```

3) Create an array and print its ndim, shape, size, and dtype

```
arr = np.array([[10, 84, 30],
                [44, 50, 61]])

print("Array:")
print(arr)

print("ndim:", arr.ndim)
print("shape:", arr.shape)
print("size:", arr.size)
print("dtype:", arr.dtype)
```

```
# ndim -> Number of dimensions  
# shape -> Number of rows and columns  
# size -> Total number of elements  
# dtype -> Data type of elements
```

```
Array:  
[[10 84 30]  
 [44 50 61]]  
ndim: 2  
shape: (2, 3)  
size: 6  
dtype: int64
```

4) Extract the last element from a 1D array.

```
arr = np.array([10, 20, 30, 40, 50])  
print(arr[-1])  
  
# or  
print(arr[arr.size - 1])  
  
# To extract the last element from a 1D NumPy array, use negative indexing.  
  
50  
50
```

5) Select the first two columns of a 2D array.

```
arr = np.array([[10, 20, 30, 40],  
               [50, 60, 70, 80],  
               [90, 100, 110, 120]])  
  
result = arr[:, :2]  
print(result)  
  
[[ 10  20]  
 [ 50  60]  
 [ 90 100]]
```

6) Reshape 12 numbers into a 3 x 4 matrix.

```
arr = np.arange(1, 13)  
result = arr.reshape(3, 4)  
print(result)
```

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]]
```

7) Transpose a 2 x 3 matrix.

```
arr = np.array([[1, 2, 3],
                [4, 5, 6]])

result = arr.T

print(result)

# or

result = np.transpose(arr)
print(result)

[[1 4]
 [2 5]
 [3 6]]
```

8) Multiply every element of an array by 5.

```
arr = np.array([10, 20, 30, 40, 50])

result = arr * 5

print(result)

[ 50 100 150 200 250]
```

9) Create a feature matrix X with 5 samples and 3 features. Print its shape.

```
X = np.array([
    [10, 20, 30],
    [15, 25, 35],
    [20, 30, 40],
    [25, 35, 45],
    [30, 40, 50]
])

print(X)
print("Shape:", X.shape)

[[10 20 30]
 [15 25 35]
 [20 30 40]
 [25 35 45]]
```

```
[30 40 50]]  
Shape: (5, 3)
```

10) Calculate the mean of each feature using axis=0

```
X = np.array([  
    [10, 20, 30],  
    [15, 25, 35],  
    [20, 30, 40],  
    [25, 35, 45],  
    [30, 40, 50]  
)  
  
mean = np.mean(X, axis=0)  
  
print(mean)  
[20. 30. 40.]
```

11) Filter all rows where the first feature is greater than 50.

```
X = np.array([  
    [60, 20, 30],  
    [40, 50, 60],  
    [75, 25, 35],  
    [30, 40, 50]  
)  
  
filtered_rows = X[X[:, 0] > 50]  
  
print(filtered_rows)  
[[60 20 30]  
 [75 25 35]]
```

12) Standardize every feature column.

```
mean = np.mean(X, axis=0)  
std = np.std(X, axis=0)  
  
X_standardized = (X - mean) / std  
  
print(X_standardized)  
[[ 0.50128041 -1.15311332 -1.15311332]  
 [-0.64450339  1.36277029  1.36277029]  
 [ 1.36061826 -0.73379939 -0.73379939]  
 [-1.21739529  0.52414242  0.52414242]]
```

13) Create a random matrix of shape (10, 4) using a fixed seed.

```
import numpy as np

np.random.seed(42)
matrix = np.random.rand(10, 4)
print(matrix)

# A random matrix means a matrix containing random numbers.
# Here, we use a fixed seed so that the same random numbers are
generated every time

[[0.37454012 0.95071431 0.73199394 0.59865848]
 [0.15601864 0.15599452 0.05808361 0.86617615]
 [0.60111501 0.70807258 0.02058449 0.96990985]
 [0.83244264 0.21233911 0.18182497 0.18340451]
 [0.30424224 0.52475643 0.43194502 0.29122914]
 [0.61185289 0.13949386 0.29214465 0.36636184]
 [0.45606998 0.78517596 0.19967378 0.51423444]
 [0.59241457 0.04645041 0.60754485 0.17052412]
 [0.06505159 0.94888554 0.96563203 0.80839735]
 [0.30461377 0.09767211 0.68423303 0.44015249]]
```

14) Calculate $X @ \text{weights}$ for a feature matrix and weight vector.

```
X = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])

weights = np.array([0.1, 4.2, 6.3], dtype=int)
result = X @ weights
print(result)

# Each row of X is multiplied with the weights, and the values are
added to get one result for each row.
# The @ operator is used for matrix multiplication.

[26 56 86]
```

15) Add a bias vector using broadcasting.

```
X = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])
bias = np.array([10, 20, 30])
result = X + bias
print(result)
```

A bias vector is added to the output of a model. NumPy automatically adds the bias to each row using broadcasting.

```
[[11 22 33]
 [14 25 36]
 [17 28 39]]
```

16) Calculate MSE between y_true and y_pred.

```
y_true = np.array([10, 30, 50, 70])
y_pred = np.array([12, 18, 29, 87])
mse = np.mean((y_true - y_pred) ** 2)
print(mse)
# MSE (Mean Squared Error) is used to find the average squared difference between the actual values -> y_true and predicted values -> y_pred.
```

219.5

17) Create a 28 x 28 grayscale image and print its shape.

```
image = np.random.randint(0, 256, (28, 28))
print(image.shape)

# A grayscale image contains only one channel where each value represents the brightness of a pixel. Here, 28 x 28 means -> we create an image with 28 rows and 28 columns.
```

(28, 28)

18) Create a batch of 64 RGB images of size 224 x 224.

```
images = np.random.randint(0, 256, (64, 224, 224, 3))
print(images.shape)

# A batch means storing multiple images together. we create 64 RGB images, where each image has a size of 224 x 224 pixels.
# In this 64 -> no.of. Images, 224, 224 -> height and width of each image, 3 -> RGB color channels (Red, Green, Blue)
```

(64, 224, 224, 3)

19) Create an array containing NaN and calculate its NaN-safe mean.

```
arr = np.array([10, 20, np.nan, 30, 40])
mean = np.nanmean(arr)
print(mean)

# NaN means Not a Number. A NaN-safe mean ignores the NaN values while
```

calculating the average
And for unsafe while cal with nan we use np.mean() -> then it give us as nan as o/p.

25.0

20) Explain the difference between $A * B$ and $A @ B$ in your own words

```
A = np.array([
    [1, 2],
    [3, 4]
])
```

```
B = np.array([
    [5, 6],
    [7, 8]
])
```

```
print("A * B:")
print(A * B)
```

```
print()
```

```
print("A @ B:")
print(A @ B)
```

*# In NumPy, * and @ are used for two different types of multiplication.*

*# A * B → element-wise multiplication. It multiplies each corresponding element.*

A @ B → matrix multiplication. It follows the rules of matrix multiplication.

```
A * B:
[[ 5 12]
 [21 32]]
```

```
A @ B:
[[19 22]
 [43 50]]
```